



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

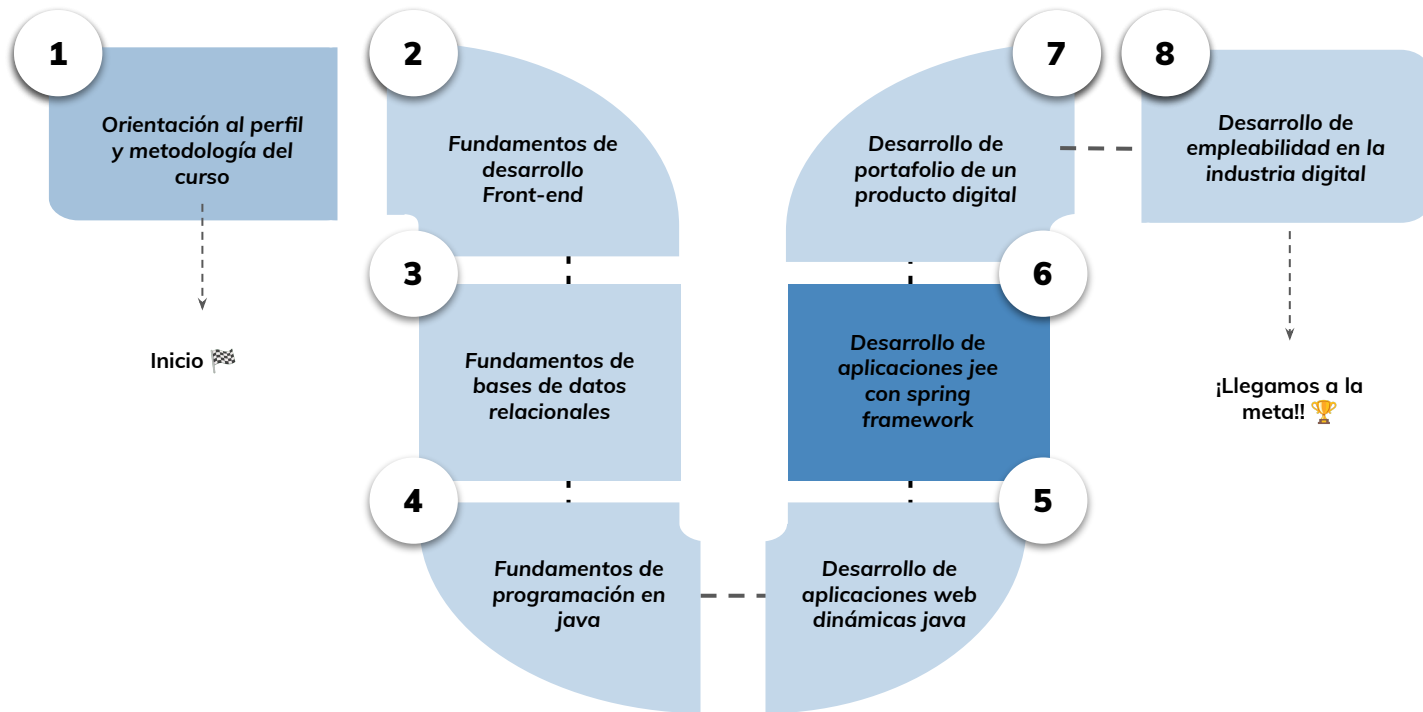


› La interoperabilidad entre los sistemas - Parte 1

Aprendizaje Esperado 5: Utilizar spring framework para la disponibilización de un servicio rest que da solución a un problema de interoperatividad.

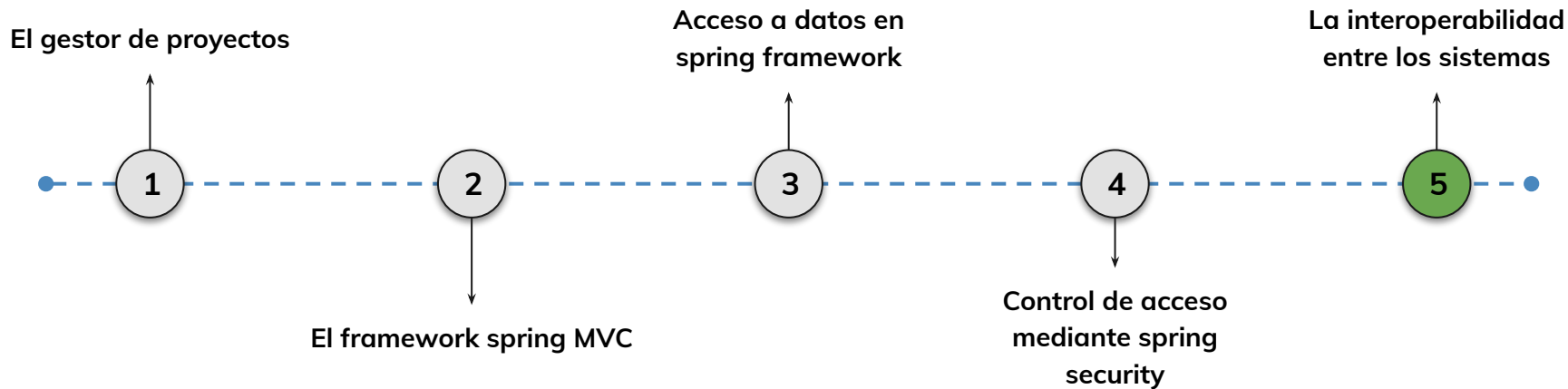
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

5.

Interoperabilidad entre sistemas con Spring REST

Exploraremos cómo el framework Spring permite construir servicios REST que faciliten la interoperabilidad entre sistemas, enfocándose en JSON como formato de intercambio.

Conceptos de Interoperabilidad

Definición y principios

Protocolos de comunicación

REST con Spring Framework

Estructura de un servicio RESTful

Implementación con JSON en Spring



Objetivos de aprendizaje

¿Qué aprenderás?



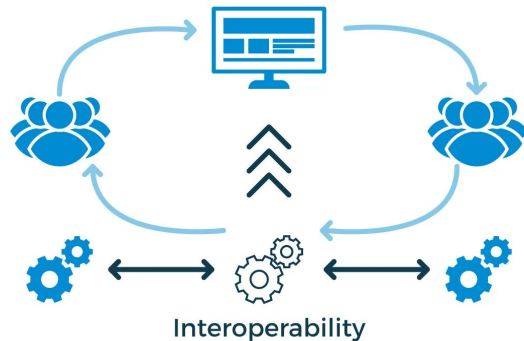
- Comprender el concepto de interoperabilidad entre sistemas.
- Identificar los principales protocolos de comunicación y datos.
- Reconocer los principios de diseño RESTful.
- Crear servicios REST con Spring Framework.
- Usar JSON como formato estándar de intercambio de datos.

➤ La interoperabilidad entre los sistemas



La interoperabilidad entre los sistemas

La interoperabilidad entre sistemas es un concepto clave en la arquitectura de software moderna. Se refiere a la **capacidad de diferentes sistemas, plataformas o aplicaciones para intercambiar datos y operar de manera conjunta de manera efectiva**. Este aspecto es crucial en un entorno tecnológico diverso, donde coexisten diferentes tecnologías, lenguajes de programación, arquitecturas y protocolos.

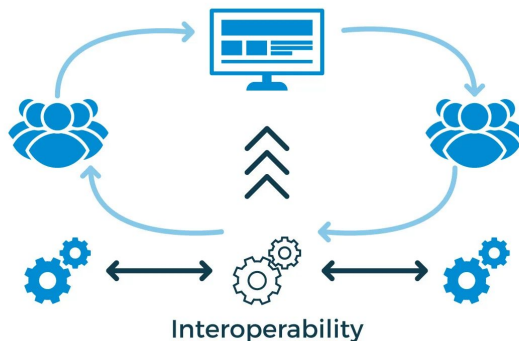




La interoperabilidad entre los sistemas

Para lograr la interoperabilidad entre sistemas es necesario que:

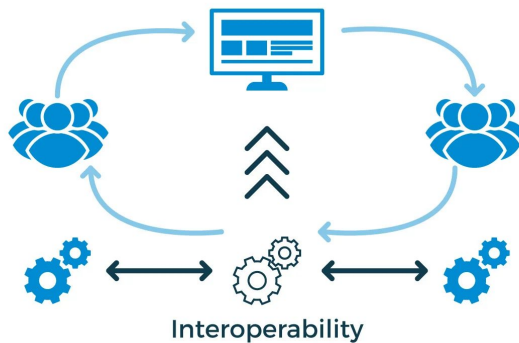
- **Hablen un lenguaje común** : esto se logra definiendo protocolos y formatos estándar para la comunicación.
- **Tengan interfaces compatibles** : las APIs de cada sistema deben poder conectarse e interactuar sin problemas.





La interoperabilidad entre los sistemas

La interoperabilidad se define como la capacidad de diferentes sistemas para trabajar juntos y compartir información de manera efectiva. Es un principio esencial en el diseño de sistemas, ya que permite la integración fluida y la comunicación entre diversas tecnologías sin perder la funcionalidad.





La interoperabilidad entre los sistemas

Importancia de la Interoperabilidad:

- **Integración sin Fricción** : Facilita la integración sin problemas de sistemas heterogéneos, permitiendo que trabajen como un todo coherente.
- **Flexibilidad y Adaptabilidad:** Facilita la incorporación de nuevas tecnologías y la adaptación a cambios en el entorno tecnológico sin afectar la funcionalidad existente.
- **Reutilización de Recursos** : Permite la reutilización eficiente de recursos, ya que los sistemas pueden utilizar componentes y servicios de otros sistemas.





La interoperabilidad entre los sistemas

Conceptos claves de la Interoperabilidad

La interoperabilidad, en el contexto de sistemas distribuidos, engloba una serie de conceptos esenciales que son fundamentales para lograr una comunicación efectiva entre diferentes componentes o sistemas. Estos conceptos clave son la piedra angular que permite que sistemas heterogéneos trabajen de manera conjunta y compartan información de manera significativa.





La interoperabilidad entre los sistemas

Conceptos clave de la Interoperabilidad

1. Sintaxis y Semántica : Estos dos conceptos trabajan en conjunto para garantizar que los sistemas comprendan la forma y el significado de los datos intercambiados, esencial para evitar malentendidos y asegurar una comunicación precisa.

- **Sintaxis** : Este concepto se refiere a la estructura y el formato de los datos intercambiados entre sistemas. En otras palabras, se trata de cómo se organizan y presentan los datos. Un ejemplo común es el formato **JSON**, que define la estructura de los datos utilizando pares **clave-valor**.
- **Semántica** : Mientras que la sintaxis se ocupa de la estructura, la semántica se centra en el significado de los datos. Incluye la **interpretación y comprensión de la información intercambiada**. Por ejemplo, en un contexto de intercambio de datos sobre productos, la semántica podría especificar que el campo "precio" representa la cantidad en la moneda local.



< > La interoperabilidad entre los sistemas

Conceptos claves de la Interoperabilidad

2. Mensajes y Paquetes:

- **Mensaje** : En el contexto de interoperabilidad, un mensaje es la unidad básica de datos enviada de un sistema a otro. Puede contener información específica y estar estructurado de acuerdo con la sintaxis y la semántica acordadas entre los sistemas. Por ejemplo, en un sistema de pedidos en línea, un mensaje podría contener detalles sobre un artículo comprado.
- **Paquete** : Los mensajes no siempre viajan solos; a menudo, se agrupan en paquetes. Un paquete es simplemente un conjunto de datos agrupados para facilitar la transmisión eficiente entre sistemas. La segmentación de información en paquetes ayuda a optimizar la velocidad y la confiabilidad de la transmisión.





La interoperabilidad entre los sistemas

Conceptos claves de la Interoperabilidad

3. **Protocolos de Comunicación** : Los protocolos de comunicación son conjuntos de reglas y convenciones que rigen la forma en que los sistemas intercambian datos. Algunos ejemplos comunes incluyen:

- **TCP/IP**
- **HTTP/HTTPS**
- **REST**
- **SOAP**





La interoperabilidad entre los sistemas

Tipos de la Interoperabilidad

4. **Arquitecturas Orientadas a Servicios** (SOA): es un enfoque arquitectónico que utiliza servicios independientes y bien definidos para lograr la interoperabilidad. Los servicios se comunican a través de estándares como SOAP y REST.

5. **Orquestación y Coreografía** :

- **Orquestación** : Control centralizado de la secuencia de ejecución de servicios en un flujo de trabajo.
- **Coreografía** : Coordinación descentralizada donde los servicios interactúan de manera autónoma sin un control central.





La interoperabilidad entre los sistemas

Ventajas de la interoperabilidad:

- **Flexibilidad y Adaptabilidad** : Facilita la integración de nuevas tecnologías, actualizaciones y cambios en el entorno tecnológico. Los sistemas interoperables son más adaptables a la evolución del panorama tecnológico.
- **Reutilización de Recursos** : Permite la reutilización eficiente de servicios, componentes y datos. Los sistemas pueden beneficiarse de los recursos existentes sin tener que reinventar la rueda.





La interoperabilidad entre los sistemas

Ventajas de la interoperabilidad:

- **Eficiencia y Mejora del Rendimiento** : Facilita la comunicación eficiente y la optimización de procesos. Los sistemas interoperables pueden operar de manera más eficiente y brindar un mejor rendimiento.
- **Reducción de la Dependencia Tecnológica** : Permite la integración de sistemas independientemente de la tecnología subyacente. Se reduce la dependencia de una tecnología específica, lo que facilita la evolución y la actualización.





La interoperabilidad entre los sistemas

Desventajas de la interoperabilidad:

- **Complejidad en la Implementación** : La implementación de interoperabilidad puede ser compleja, especialmente en entornos con tecnologías diversas. Requiere una planificación cuidadosa y la gestión de la complejidad técnica.
- **Seguridad y Gobernanza** : La interoperabilidad mal gestionada puede plantear riesgos de seguridad. Es necesario establecer políticas de gobernanza y aplicar prácticas de seguridad robustas.





La interoperabilidad entre los sistemas

Desventajas de la interoperabilidad:

- **Compatibilidad de Estándares** : La variabilidad en la adopción de estándares puede causar problemas de compatibilidad. Se deben seleccionar estándares comunes y respaldados por la industria para evitar conflictos.
- **Costos de Implementación** : La implementación y mantenimiento de la interoperabilidad pueden generar costos significativos. Se debe equilibrar la inversión con los beneficios esperados y la escalabilidad a largo plazo.





La interoperabilidad entre los sistemas

Consideraciones adicionales para mejores prácticas:

- **Documentación Exhaustiva** : Documenta claramente los protocolos, formatos de datos y puntos finales para facilitar la integración.
- **Pruebas Rigurosas**: Implementa pruebas unitarias y de integración para garantizar la interoperabilidad en diversas condiciones.
- **Gestión de Versiones**: Establece prácticas de gestión de versiones para evitar conflictos y asegurar la compatibilidad.
- **Monitoreo y Resolución de Problemas**: Implementa herramientas de monitoreo para identificar y resolver problemas de interoperabilidad de manera proactiva.





La interoperabilidad entre los sistemas

La interoperabilidad entre sistemas es esencial en el panorama tecnológico actual, donde la diversidad tecnológica es la norma. Aunque presenta desafíos, los beneficios en términos de **flexibilidad**, **eficiencia** y **reutilización** de recursos hacen que la inversión en interoperabilidad sea valiosa. Al seguir mejores prácticas, gestionar la complejidad y adaptarse a estándares comunes, las organizaciones pueden construir mejores ecosistemas tecnológicos que impulsan la innovación y el crecimiento continuo.



➤ **Protocolos de intercambio de datos**



Protocolos de intercambio de datos

Navegando por el Mundo de la Comunicación Digital

Los protocolos de intercambio de datos son los lenguajes fundamentales que permiten la comunicación entre sistemas distribuidos. Estos protocolos definen las reglas y el formato para el intercambio de información, facilitando la interoperabilidad y la transmisión de datos entre aplicaciones y dispositivos.

La elección del protocolo de intercambio de datos es fundamental para el éxito de la interoperabilidad entre sistemas. Veamos algunos de los protocolos más comunes, examinando sus características, aplicaciones y ventajas.

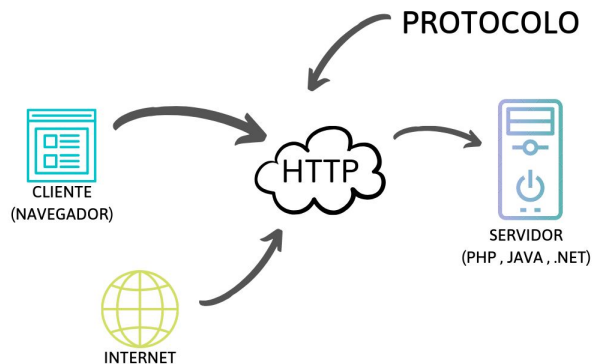




Protocolos de intercambio de datos

1- HTTP (Protocolo de Transferencia de Hipertexto): es el protocolo fundamental para la transferencia de datos en la World Wide Web. Opera sobre el modelo cliente-servidor, donde el cliente realiza solicitudes y el servidor responde con los recursos solicitados.

Es ampliamente utilizado en aplicaciones web para la transferencia de documentos, imágenes y otros recursos.





Protocolos de intercambio de datos

HTTP: Características Principales

- Utiliza los métodos estándar como GET (obtener), POST (enviar datos al servidor), PUT (actualizar) y DELETE (eliminar).
- Es un protocolo sin estado, lo que significa que cada solicitud del cliente es independiente y no mantiene información sobre solicitudes anteriores.
- Simple y fácil de implementar.
- Compatible con una variedad de tipos de datos.

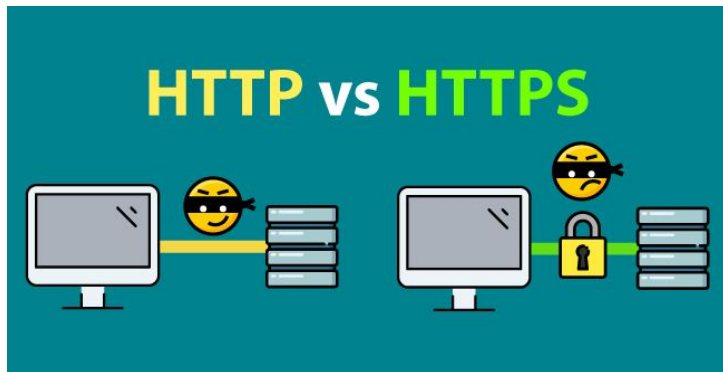




Protocolos de intercambio de datos

2- HTTPS (HTTP Seguro): **HTTPS es la versión segura de HTTP que utiliza capas de cifrado**, generalmente SSL/TLS, para garantizar la seguridad de la comunicación entre el cliente y el servidor.

Es crucial para aplicaciones que manejan información sensible, como datos de tarjetas de crédito o contraseñas.





Protocolos de intercambio de datos

HTTPS Características Principales:

- Utiliza certificados digitales para autenticar la identidad del servidor.
- Proporciona cifrado de extremo a extremo para proteger la privacidad de los datos.
- Asegura la confidencialidad y la integridad de los datos transmitidos.

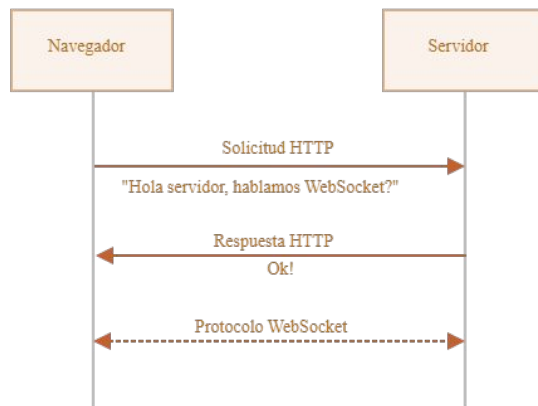




Protocolos de intercambio de datos

3- WebSocket: es un protocolo de comunicación bidireccional que permite la comunicación en tiempo real entre un cliente y un servidor. A diferencia de HTTP, WebSocket establece una conexión persistente, lo que permite una comunicación más eficiente y de baja latencia.

Principalmente utilizado en aplicaciones que requieren actualizaciones en tiempo real, como chats en línea y transmisiones en vivo.





Protocolos de intercambio de datos

WebSocket: Características Principales

- Permite una comunicación bidireccional, lo que significa que tanto el cliente como el servidor pueden enviar mensajes en cualquier momento.
- Reduce la sobrecarga de la comunicación al eliminar la necesidad de establecer una conexión nueva para cada solicitud.
- Mayor eficiencia en comparación con los enfoques tradicionales basados en HTTP.



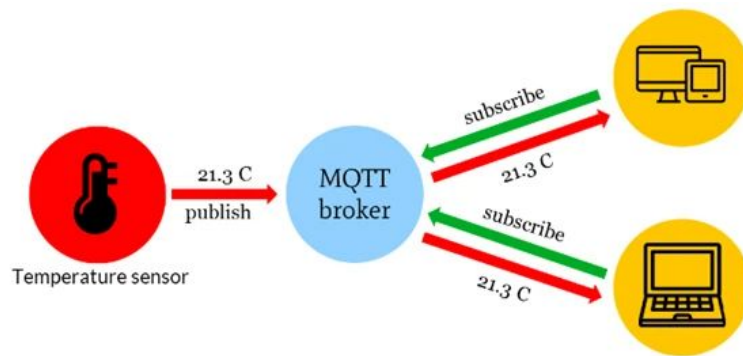


Protocolos de intercambio de datos

4- MQTT (Protocolo de Telemetría de Mensajería y Calidad de Servicio Mínimo):

Es un protocolo ligero de mensajería diseñado para situaciones de baja capacidad y alta latencia, siendo ideal para el Internet de las cosas (IoT).

Ampliamente utilizado en entornos IoT para la transmisión eficiente de datos entre dispositivos.





Protocolos de intercambio de datos

MQTT: Características Principales

- Utiliza un modelo de publicación/suscripción donde los dispositivos pueden publicar mensajes en "temas" a los que otros dispositivos pueden suscribirse.
- Ofrece diferentes niveles de calidad de servicio (QoS) para garantizar la entrega de mensajes.
- Eficiente en términos de ancho de banda y recursos.





Protocolos de intercambio de datos

5- XML (Lenguaje de Marcado Extensible): es un lenguaje de marcado que permite la representación estructurada de datos mediante el uso de etiquetas. Es ampliamente utilizado para el intercambio de datos entre sistemas heterogéneos.

Comúnmente utilizado en la representación de datos en documentos y configuraciones.





Protocolos de intercambio de datos

XML: Características Principales

- Define reglas para la creación de documentos estructurados mediante el uso de etiquetas anidadas.
- Es legible por humanos y fácil de entender.
- Versátil y extensible, lo que permite la representación de datos complejos.

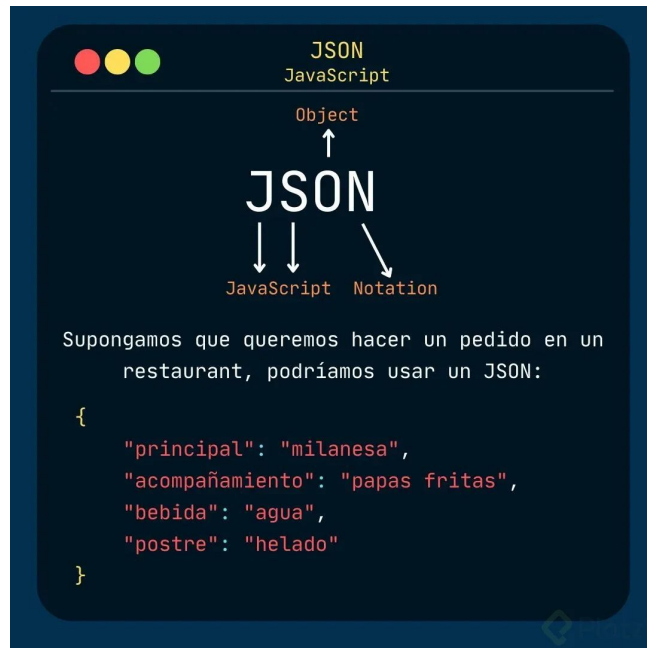




Protocolos de intercambio de datos

6- JSON (Notación de Objetos de JavaScript): es un formato ligero de intercambio de datos que utiliza una sintaxis similar a la de los objetos en JavaScript. Es fácilmente legible por máquinas y humanos.

Ampliamente utilizado en servicios web y aplicaciones que requieren intercambio de datos eficiente.





Protocolos de intercambio de datos

6- **JSON** Características Principales:

- Utiliza pares clave-valor para representar datos de manera estructurada.
- Es más compacto que XML y fácil de analizar.
- Ampliamente utilizado en servicios web y aplicaciones que requieren intercambio de datos eficiente.
- Ligero y fácil de procesar, especialmente en entornos web.

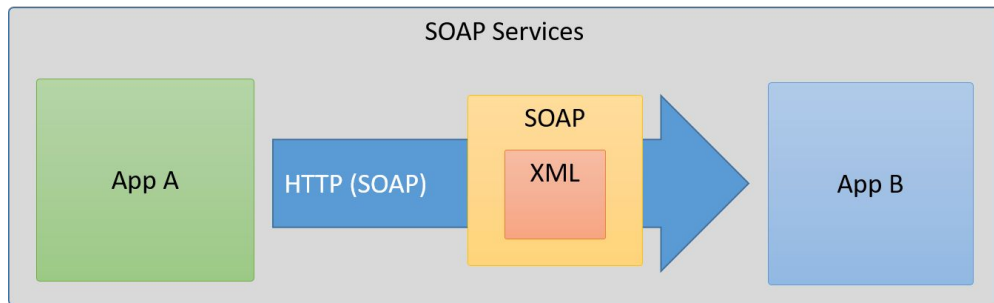




Protocolos de intercambio de datos

7- SOAP (Protocolo Simple de Acceso a Objetos): es un protocolo basado en **XML** utilizado para el intercambio de mensajes estructurados entre sistemas. Se centra en la definición de reglas para la estructura de mensajes y se utiliza comúnmente en entornos donde se requiere una comunicación más formal y controlada.

Ampliamente utilizado en servicios web y aplicaciones empresariales donde se necesita una comunicación más estructurada.





Protocolos de intercambio de datos

SOAP Características Principales:

- Define una estructura de mensaje rica en XML con encabezados y cuerpo.
- Utiliza protocolos de transporte como HTTP y SMTP para la transmisión.
- Ampliamente utilizado en servicios web y aplicaciones empresariales donde se necesita una comunicación más estructurada.
- Permite la comunicación entre aplicaciones independientes a través de estándares bien definidos.
- Proporciona un nivel de seguridad y control adicional.

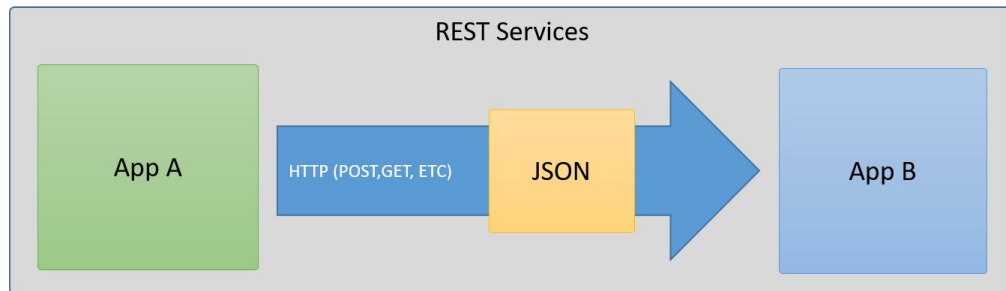




Protocolos de intercambio de datos

8- REST (Transferencia de Estado Representacional): es un estilo arquitectónico que se basa en la representación de recursos y opera sobre el protocolo **HTTP**. Se centra en la simplicidad y la escalabilidad, utilizando operaciones estándar HTTP para realizar acciones sobre recursos.

Ampliamente utilizado en servicios web y aplicaciones que requieren una comunicación flexible y liviana.





Protocolos de intercambio de datos

REST Características Principales:

- Utiliza los métodos HTTP estándar (GET, POST, PUT, DELETE) para operaciones CRUD en recursos.
- Es un enfoque sin estado, lo que significa que cada solicitud del cliente contiene toda la información necesaria para entender y procesar la solicitud.
- Favorece la simplicidad y la escalabilidad.
- Permite la construcción de APIs web eficientes y fáciles de consumir.





Protocolos de intercambio de datos

Los protocolos de intercambio de datos cumplen varias funciones importantes:

- Definen un formato estándar para representar los datos, como XML, JSON, CSV, etc. Esto permite que sistemas distintos entiendan la información.
- Establecen cómo debe realizarse la comunicación: solicitud, respuesta, secuenciación, etc. Por ejemplo, el protocolo HTTP define verbos como GET, POST, PUT.
- Permiten la detección y corrección de errores. Los protocolos pueden especificar códigos de error y solicitudes de retransmisión.
- Proporcionan seguridad, como encriptación y autenticación. Por ejemplo, HTTPS es HTTP sobre SSL.
- Facilitan la integración al actuar como un "lenguaje común". Los sistemas solo necesitan "hablar" ese protocolo para entenderse.



➤ Principios REST



Principios REST

1- Separación cliente-servidor

Los sistemas REST **tienen una separación explícita** entre las preocupaciones del cliente y las del servidor.

- **Los clientes no se preocupan por el almacenamiento** de datos o los detalles de implementación del servidor: sólo necesitan saber cómo interactuar con las capacidades del servidor a través de la interfaz.
- **El servidor no se ocupa de las capacidades del cliente** ni de los detalles de implementación: sólo necesita saber cómo satisfacer las peticiones y servir las respuestas correspondientes.

Esto permite que las arquitecturas de servidor y cliente evolucionen de forma independiente **sin un fuerte acoplamiento**.





Principios REST

2- Sin estado

En la web, HTTP es un protocolo sin estado: **el servidor no almacena el estado del cliente entre peticiones.** REST se basa en este concepto y separa aún más el estado del cliente del estado del servidor para aumentar la escalabilidad.

- **El servidor no necesita gestionar sesiones de clientes** , sólo gestionar peticiones individuales de forma aislada. Si el contexto del cliente es necesario para gestionar una solicitud, **el cliente proporciona el contexto con cada solicitud individual** .





Principios REST

3- Almacenamiento en caché

REST pretende aprovechar una de las principales ventajas del uso de HTTP: el almacenamiento en caché **a través de las cabeceras HTTP y los códigos de estado** .

- **Los recursos son identificables únicamente a través de URL** , sin que el contexto del cliente se almacene en el servidor. Esto permite almacenar en caché las respuestas, ya que la misma URL produce la misma respuesta.
- **No es necesario gestionar sesiones de cliente en el servidor** : basta con almacenar en caché las respuestas anteriores para mejorar el rendimiento y la escalabilidad.





Principios REST

4- Sistema por capas

La arquitectura de un sistema REST se compone de capas jerárquicas por función o propósito. Esto permite el equilibrio de carga, el almacenamiento en caché compartido, las capas de seguridad y mejora la flexibilidad y la escalabilidad.

- El nivel de presentación o **capa de interfaz de usuario** se encarga de la interfaz con el usuario final.
- Por debajo de ella, la **capa de aplicación** se encarga de la lógica del proceso funcional y de la validación.
- Por debajo, el **nivel de datos** se encarga del almacenamiento y el acceso a bases de datos o sistemas de archivos.

La capa cliente no necesita conocer todas las capas del sistema, solo la capa inmediata con la que interactúa.





Principios REST

5- Interfaz uniforme

REST pretende obtener ventajas basándose en el estándar HTTP existente en lugar de crear nuevas pilas de protocolos. Mediante el uso de verbos HTTP, URL, cabeceras y códigos de estado, REST **aprovecha la sintaxis existente del protocolo HTTP en lugar de inventar nuevos estándares basados en XML** .

Esto simplifica la arquitectura general del sistema para ofrecer un modelo de desarrollo unificado en todos los sistemas y la web.



➤ Recursos REST



Recursos REST

La pieza central de REST son los recursos abstractos que modelan objetos de información, datos, entidades, o conceptos específicos. Un recurso es cualquier cosa que pueda ser nombrada e identificada individualmente. Por ejemplo:

- Un documento o archivo
- Un objeto de datos relacional como una fila en una base de datos
- Un servicio de procesamiento específico
- Una colección o contenedor de otros recursos
- Una entidad conceptual no textual





Recursos REST

Cada recurso tiene una identidad única y permanente definida por un identificador universal, típicamente un **URI** (Uniform Resource Identifier).

Por ejemplo, un recurso que modela el concepto de "cliente" en un dominio empresarial determinado puede identificarse de forma única como:

<http://api.domain.com/customers/1234>





Recursos REST

Manipulación de recursos a través de representación

Los clientes no acceden directamente a los recursos mismos, sino a una representación temporal de su estado capturado en cierto momento. Las representaciones transfieren el estado en formatos estandarizados como JSON o XML conteniendo atributos de datos u otros metadatos.





Recursos REST

Manipulación de recursos a través de representación

Por ejemplo, la representación de JSON para el recurso de cliente anterior puede ser:

```
{  
  "id": 1234,  
  "name": "Juan Perez",  
  "address": "Calle 123"  
}
```

Esto desacopla el recurso en sí del formato de representación específico. Incluso las interacciones en medios no textuales como video o audio se consideran "representaciones" de recurso.





Recursos REST

Accediendo recursos a través de identificadores

Los clientes acceden, manipulan y transfieren estas representaciones de recursos de forma uniforme a través de sus identificadores estándares. Esto típicamente utiliza URIs HTTP estándar en combinación con métodos HTTP estándar como GET, POST, PUT, DELETE que manipulan el recurso de forma predecible.





Recursos REST

Accediendo recursos a través de identificadores

Las URIs identifican y ubican recursos, pero también determinan acción esperada cuando se accede a través de un verbo HTTP:

```
GET http://api.domain.com/customers/1234  
PUT http://api.domain.com/customers/1234  
DELETE http://api.domain.com/customers/1234
```

Estos patrones determinan comportamiento uniforme estándar sin más especificación formal gracias a construcciones web existentes.





Seguridad en REST

Seguridad en REST

Como REST opera sobre el protocolo HTTP existente, puede implementar los mismos mecanismos de seguridad subyacentes:

- SSL/TLS para cifrado de tráfico
- Autenticación HTTP para validar usuarios
- Tokens OAuth para delegación de acceso
- Listas de control de acceso para autorizar recursos
- Políticas de uso para prevenir abusos

Al aprovechar totalmente credenciales y controles de seguridad HTTP, no se requieren mecanismos inventados personalizados para proteger una API REST.



› **Ventajas y Desventajas de REST**



Ventajas de REST

REST y sus principios guía ofrecen varias ventajas clave sobre otros estilos arquitectónicos web:

1- **Desacoplamiento mejorado** : La separación de preocupaciones entre cliente y servidor sin estado compartido mejora modularidad. Los clientes solo conocen URIs de recursos, pero no conocen la implementación del servidor. Esta abstracción desacopla frontends y backends, permitiendo evolución independiente.

2- **Simplicidad inherente** : REST aprovecha standards web asistentes como URIs, HTTP y JSON que son simples, ubicuos y familiares. No se requieren nuevas convenciones o regímenes complejos.





Ventajas de REST

3- **Entrega optimizada de recursos** : Las URIs REST desreferencian recursos grandes permitiendo entrega en etapas. Las cachés y redes de distribución de contenido optimizan entrega utilizando mecanismos web sin cambios.

4- **Rendimiento y escalabilidad** : Sin estado y almacenamiento en caché inherentes permiten escalar horizontalmente. Los servidores REST simples procesan más solicitudes, más rápido y con menos recursos que los servicios RPC encapsulados.

5- **Portabilidad y longevidad** : REST es independiente del lenguaje y la plataforma, permitiendo integrarse ampliamente entre ecosistemas y generaciones de tecnología a medida que evolucionan.





Desventajas de REST

Sin embargo, REST también tiene debilidades inherentes a considerar:

- 1- **Complejidad inicial de modelado de recursos** : Modelar todas las capacidades como recursos con representaciones transferibles requiere análisis de dominio avanzado. Es más difícil que simplemente exponer objetos y métodos.
- 2- **No funciona bien para llamadas multidominio** : REST para un solo recurso funciona bien. Pero las operaciones que abarcan múltiples recursos requieren abstracción adicional para coordinar confirmaciones transaccionales.
- 3- **Rigidez en URLs difícil de iterar** : Las URLs que identifican recursos se vuelven frágiles ante cambios. Agregar nuevos parámetros o estructuras de datos requiere nuevas URLs que pueden interferir con el cliente.





Desventajas de REST

Por lo tanto, si bien REST ofrece ventajas en caching, desacoplamiento, longevidad de APIs y escalamiento horizontal, también viene con un mayor esfuerzo inicial de diseño y limitaciones operativas. Los ingenieros deben sopesar compromisos al elegir un estilo arquitectónicos por sobre otros.

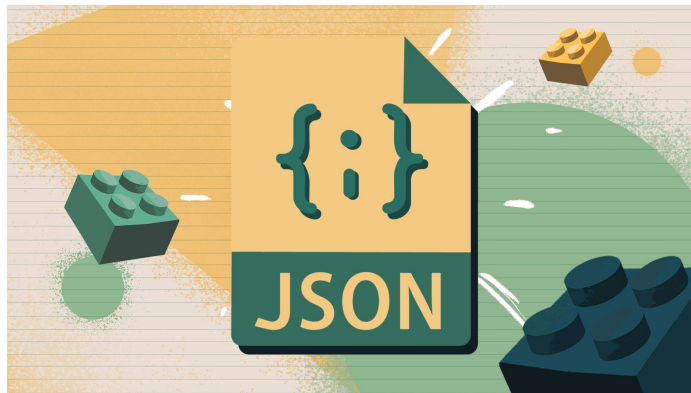


› JSON



JSON

JSON, que significa "**J**ava**S**cript **O**bject **N**otation", es un formato de intercambio de datos ligero y legible por humanos. Fue derivado originalmente del lenguaje de programación JavaScript, pero se ha convertido en un formato independiente que es ampliamente utilizado en el intercambio de datos en la web y en aplicaciones.





JSON

La notación JSON se caracteriza por utilizar una estructura de pares **clave-valor** . Los datos se organizan en objetos, arrays, números, cadenas, booleanos y valores nulos. La simplicidad de su estructura permite la fácil interpretación por parte de humanos y máquinas.

Se basa en dos estructuras de datos principales: **objetos** y **arrays** .





JSON

Un **objeto** en JSON es una colección **no ordenada** de pares clave-valor. Cada clave debe ser una cadena de caracteres y debe ser única dentro del objeto. Los valores pueden ser strings, números, booleanos, null, objetos o arrays.

```
{  
  "nombre": "Juan",  
  "edad": 30,  
  "casado": false,  
  "hijos": null,  
  "direcciones": {  
    "casa": "123 Calle Principal",  
    "trabajo": "456 Calle Secundaria"  
  },  
  "hobbies": ["lectura", "cine", "viajes"]  
}
```





JSON

Un **array** es una secuencia ordenada de valores. Los valores pueden ser de cualquier tipo de datos JSON, incluyendo objetos y otros arrays.

```
["manzana", "naranja", "banana", {"fruta": "uva"}, [1, 2, 3]]
```





JSON

Algunas **diferencias** notables en contraste **con XML** son:

- **Sintaxis** : JSON tiene una sintaxis **más concisa y fácil** de leer en comparación con XML. Las etiquetas en XML pueden hacer que los documentos sean más verbosos.
- **Peso del Documento** : JSON **tiende a ser más ligero** porque tiene menos caracteres de marcado que XML, lo que resulta en una transmisión de datos más eficiente.
- **Amplia Adopción en la Web** : JSON es el formato de datos nativo para JavaScript, lo que facilita su manipulación en aplicaciones web. Además, es ampliamente aceptado en muchas otras plataformas y lenguajes de programación.
- **Facilidad de Mapeo a Estructuras de Datos en Programación** : La estructura de JSON se mapea fácilmente a estructuras de datos en muchos lenguajes de programación, simplificando la integración de datos en aplicaciones.



› JSON y Spring



JSON y Spring

La utilización de JSON en entornos de desarrollo Java, especialmente en el marco de trabajo Spring, ofrece diversas **ventajas** que van más allá de simplemente facilitar el intercambio de datos. Las principales son:

1. **Ligereza y Eficiencia**
2. **Interoperabilidad y Flexibilidad**
3. **Facilidad de Lectura y Depuración**





JSON y Spring

Java es conocido por ser un lenguaje de programación robusto y orientado a objetos, pero en entornos de desarrollo modernos, la eficiencia en el intercambio de datos es esencial. Aquí es donde JSON brilla, y una de las principales ventajas radica en su ligereza.

- **Estructura de Datos Compacta y Fácil de Procesar** : JSON utiliza una estructura de datos simple que es fácil de entender y procesar tanto para las aplicaciones como para los desarrolladores. En comparación con otros formatos de intercambio de datos más verbosos, como XML, JSON tiende a ser más compacto, lo que resulta en una transmisión de datos más eficiente.





JSON y Spring

- **Deserialización Eficiente en Java con Bibliotecas JSON** : JSON se integra naturalmente con bibliotecas de deserialización en Java, como Jackson y Gson, que permiten convertir datos JSON en objetos Java de manera rápida y eficiente.

```
// Ejemplo de deserialización con Jackson en Spring
String jsonString = "{\"nombre\": \"Juan\", \"edad\": 30, \"casado\": false}";
ObjectMapper objectMapper = new ObjectMapper();
Persona persona = objectMapper.readValue(jsonString, Persona.class);
```

En este ejemplo, la clase Persona puede tener campos que coincidan directamente con las claves en el objeto JSON, simplificando el proceso de deserialización.





JSON y Spring

- **Menor Consumo de Ancho de Banda y Recursos** : La ligereza de JSON se traduce en un menor consumo de ancho de banda al transmitir datos a través de la red. Esto es especialmente beneficioso en entornos donde la eficiencia de recursos es crucial, como en aplicaciones móviles o en sistemas distribuidos.
- **Rendimiento Optimizado en Aplicaciones Spring** : La facilidad con la que los datos JSON pueden ser convertidos a objetos Java facilita la manipulación de datos, mejorando la eficiencia y la capacidad de respuesta de las aplicaciones.





JSON y Spring

- **Compatibilidad con Diversos Lenguajes y Plataformas** : Una de las ventajas fundamentales de JSON es su capacidad para ofrecer interoperabilidad entre diferentes lenguajes y plataformas. Esto se alinea perfectamente con la naturaleza políglota de las aplicaciones modernas, donde diversas tecnologías colaboran para crear sistemas complejos.
- **Integración Natural con Tecnologías Web** : En el contexto de Java y Spring, la integración de JSON es especialmente valiosa en aplicaciones web. La mayoría de los navegadores web y clientes HTTP admiten nativamente la serialización y deserialización de datos JSON, lo que facilita la comunicación entre el frontend y el backend de una aplicación.





JSON y Spring

- **Flexibilidad en la Representación de Datos** : Puedes modelar datos complejos, anidando objetos y arrays, lo que facilita la representación de relaciones y estructuras de datos más avanzadas.
- **Integración con Servicios Web y APIs** : En aplicaciones Spring que interactúan con servicios web y APIs, JSON se ha convertido en el formato dominante para el intercambio de datos. La mayoría de los servicios web modernos ofrecen endpoints que aceptan y devuelven datos en formato JSON, lo que simplifica la integración de diferentes componentes de software.





JSON y Spring

- **Legibilidad para Desarrolladores y Máquinas** : Una de las características distintivas de JSON es su legibilidad tanto para los desarrolladores como para las máquinas. La sintaxis simple y clara facilita la comprensión de la estructura de los datos, lo que es esencial durante el desarrollo y la depuración de aplicaciones.
- **Herramientas de Visualización y Depuración Integradas** : En entornos de desarrollo Java, como Eclipse o IntelliJ, las herramientas de visualización y depuración están optimizadas para trabajar con datos JSON. Los desarrolladores pueden inspeccionar y visualizar fácilmente los objetos Java generados a partir de datos JSON durante la ejecución de la aplicación.





JSON y Spring

- **Facilidad en la Creación de Datos de Prueba:** Durante el desarrollo y las pruebas, la capacidad de representar datos de manera legible y fácil con JSON facilita la creación de datos de prueba realistas. Los desarrolladores pueden crear fácilmente conjuntos de datos complejos y variados para probar diferentes escenarios de aplicación.

La elección de JSON en el desarrollo de aplicaciones Spring y Java no solo es una cuestión de conveniencia, sino que también contribuye a la eficiencia, la legibilidad del código y la capacidad de respuesta de las aplicaciones. La capacidad de representar datos de manera clara y eficiente es esencial en el panorama actual del desarrollo de software.



➤ **JSON: Casos de Uso**



JSON: Casos de Uso

La adopción generalizada de JSON en el desarrollo de aplicaciones Spring y Java no solo se basa en sus ventajas teóricas, sino también en su aplicabilidad práctica en una variedad de escenarios del mundo real.

Dos típicos casos de uso en los cuales JSON desempeña un papel crucial son: **Servicios Web RESTful** y **Aplicaciones de Una Página** (SPA).

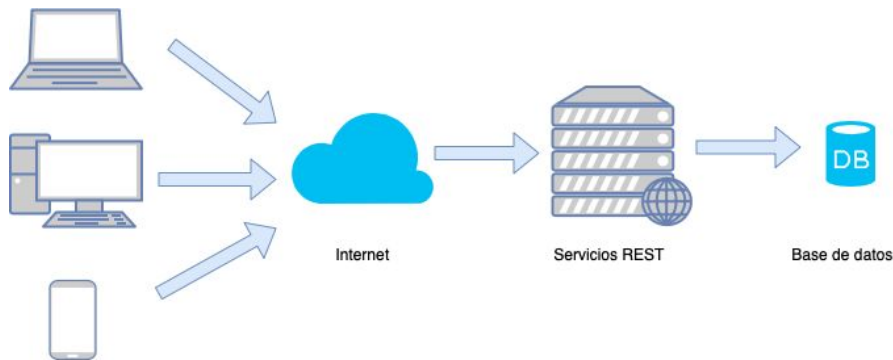




JSON: Casos de Uso

1- Servicios Web RESTful

Los servicios web RESTful (Representational State Transfer) se han convertido en el estándar predominante para el desarrollo de APIs en la arquitectura web. Estos servicios utilizan los principios fundamentales de REST para facilitar la comunicación entre sistemas distribuidos a través de HTTP. JSON, con su estructura de datos liviana y fácilmente serializable, se integra perfectamente en la arquitectura RESTful.





JSON: Casos de Uso

Intercambio de Datos en Formato JSON en Servicios RESTful

En un servicio web RESTful, la representación de recursos se realiza comúnmente en formato JSON. Los recursos, que pueden ser objetos complejos o colecciones de datos, se transmiten entre el cliente y el servidor en el cuerpo de las solicitudes y respuestas HTTP. La simplicidad y la legibilidad de JSON hacen que sea fácil para los desarrolladores entender y manipular estos datos en el proceso de desarrollo y depuración.





JSON: Casos de Uso

Intercambio de Datos en Formato JSON en Servicios RESTful

En un servicio web RESTful, la representación de recursos se realiza comúnmente en formato JSON. Los recursos, que pueden ser objetos complejos o colecciones de datos, se transmiten entre el cliente y el servidor en el cuerpo de las solicitudes y respuestas HTTP. La simplicidad y la legibilidad de JSON hacen que sea fácil para los desarrolladores entender y manipular estos datos en el proceso de desarrollo y depuración.

```
// Ejemplo de respuesta JSON en un servicio RESTful
{
  "id": 1,
  "titulo": "Artículo de Ejemplo",
  "contenido": "Este es un artículo de ejemplo sobre el uso de JSON"
}
```





JSON:Casos de Uso

Deserialización de Datos en el Servidor Spring

En el lado del servidor, en un entorno Spring, la deserialización de datos JSON se simplifica mediante bibliotecas como Jackson. Los controladores de Spring pueden recibir datos JSON en el cuerpo de las solicitudes y automáticamente deserializarlos en objetos Java, facilitando el procesamiento de estos datos en la lógica del servidor.

```
// Controlador Spring que maneja una solicitud POST con datos JSON
@RestController
@RequestMapping("/articulos")
public class ArtículoController {

    @PostMapping
    public ResponseEntity<String> crearArticulo(@RequestBody Artículo nuevoArticulo) {
        // Lógica para crear un nuevo artículo
        return ResponseEntity.ok("Artículo creado con éxito");
    }
}
```





JSON:Casos de Uso

Implementación de Versionamiento de API con JSON

En servicios web RESTful, la evolución de la API es un factor crítico. JSON facilita la implementación de versionamiento de API, donde nuevas versiones de los recursos pueden introducirse sin romper la compatibilidad con las versiones anteriores. Los clientes pueden adaptarse a cambios incrementales en la estructura de los datos JSON sin interrupciones significativas.





JSON:Casos de Uso

2- El Auge de las Aplicaciones de Una Página (SPA)

Las Aplicaciones de Una Página (SPA) se han vuelto populares en el desarrollo web moderno debido a su capacidad para proporcionar experiencias de usuario fluidas y rápidas al cargar solo una página HTML y actualizar dinámicamente el contenido. JSON desempeña un papel fundamental en estas aplicaciones al facilitar la transferencia eficiente de datos entre el cliente y el servidor.





JSON: Casos de Uso

Manejo de Navegación y Estado con JSON

En una SPA, la navegación entre diferentes vistas y el manejo del estado de la aplicación se gestionan de manera dinámica. JSON se utiliza para representar y transmitir información sobre el estado de la aplicación, lo que facilita el manejo del enrutamiento y la persistencia del estado del cliente.





JSON:Casos de Uso

Validación de Datos JSON en Spring

La validación de datos es un aspecto crucial en cualquier aplicación, y Spring proporciona mecanismos sólidos para validar datos JSON. Utilizando anotaciones como `@Valid` y `@RequestBody`, es posible validar automáticamente los datos JSON recibidos en los controladores de Spring.

```
@PostMapping
public ResponseEntity<String> crearUsuario(@Valid @RequestBody Usuario nuevoUsuario) {
    // Lógica para crear un nuevo usuario
    return ResponseEntity.ok("Usuario creado con éxito");
}
```

En este ejemplo, la anotación `@Valid` activa la validación del objeto `Usuario` según las reglas definidas en su clase.





JSON: Casos de Uso

Integración con Spring Data y Bases de Datos NoSQL

Spring Data proporciona soporte para la integración con bases de datos NoSQL, y JSON se adapta naturalmente a este entorno. Bases de datos NoSQL como MongoDB almacenan datos en formato BSON (Binary JSON), lo que facilita la integración directa con objetos JSON en el código de la aplicación.

```
// Ejemplo de entidad MongoDB utilizando Spring Data
@Document(collection = "usuarios")
public class Usuario {

    @Id
```

En este ejemplo, la entidad Usuario se almacena en una colección MongoDB, y los campos de la entidad se mapean directamente a campos JSON en la base de datos



Live Coding

¿En qué consistirá la Demo?

Vamos a desarrollar un servicio REST con Spring Boot que permita exponer información de productos y recibir órdenes en formato JSON. Esto mostrará cómo implementar interoperabilidad entre aplicaciones usando herramientas modernas y gratuitas.

1. Iniciar un proyecto Spring Boot desde Spring Initializr
2. Agregar las dependencias: Spring Web y DevTools
3. Crear la clase modelo Producto con atributos básicos
4. Crear la clase modelo OrdenEntrega para recibir datos en JSON
5. Implementar el controlador REST con endpoints GET y POST
6. Probar la API utilizando Hoppscotch.io sin necesidad de cuenta
7. Validar el formato JSON de entrada/salida y la interoperabilidad entre cliente-servidor
8. Explicar cómo este enfoque facilita la integración entre sistemas heterogéneos

Tiempo: 15 minutos



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Construyendo un Servicio REST interoperable

Construyendo un Servicio REST interoperable

Contexto: 🙌

Una empresa de logística necesita integrar su sistema de inventario con una nueva aplicación móvil externa que gestionará las entregas. Para lograrlo, requiere un servicio REST que exponga información de los productos en formato JSON y permita recibir nuevas órdenes de entrega desde otras plataformas.

Consigna: 📝

Implementa un microservicio REST utilizando Spring Boot que permita:

- Obtener la lista de productos en inventario.
- Registrar nuevas órdenes de entrega.

Toda la comunicación debe realizarse utilizando JSON para asegurar la interoperabilidad.

Tiempo 🕒: 40 minutos

Paso a paso: ⚙️

1. Crear un proyecto en Spring Initializr sin registro.
2. Agregar dependencias: Spring Web, Spring Boot DevTools.
3. Crear clase modelo Producto con id, nombre, stock.
4. Crear clase modelo OrdenEntrega con id, productoid, cantidad.
5. Implementar controlador REST con:
 - a. GET /productos → Retorna lista en JSON.
 - b. POST /ordenes → Recibe orden en JSON y muestra confirmación.
6. Probar endpoints con Hoppscotch (herramienta gratuita y sin cuenta).
7. Verificar el formato JSON y validar la interoperabilidad.

¿Alguna consulta?



Resumen

¿Qué logramos en esta clase?

- ✓ **Comprendimos la importancia de la interoperabilidad.**
- ✓ **Identificamos los principales protocolos como HTTP, JSON y REST.**
- ✓ **Estudiamos los principios RESTful aplicados en Spring.**
- ✓ **Implementamos un servicio REST básico interoperable.**
- ✓ **Usamos herramientas gratuitas como Spring Initializr y Hoppscotch.**
- ✓ **Practicamos la creación y el consumo de APIs REST.**
- ✓ **Validamos el uso de JSON para intercambiar datos entre sistemas.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son claves para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compártela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

